

Unit 2 – Software Process

Topics covered

- ✧ Software Process and operation procedure
- ✧ Process activities
- ✧ Software process models
- ✧ Coping with change

The software process

- ✧ A structured set of **activities** required to develop a software system.
- ✧ Many **different** software processes but **all involve**:
 - **Specification** – defining what the system should do;
 - **Design and implementation** – defining the organization of the system and implementing the system;
 - **Validation** – checking that it does what the customer wants;
 - **Evolution** – changing the system in response to changing customer needs.
- ✧ A **software process model** is an abstract representation of a process. It presents a description of a process from some particular perspective.

ISO/IEC/IEEE
15288-2023:
System Life
Cycle Processes

ISO/IEC/IEEE
12207-2026:
Software Life
Cycle Processes

Software Life Cycle Processes		
Agreement Processes	Technical Management Processes	Technical Processes
Acquisition Process (6.1.1)	Project Planning Process (6.3.1)	Business or Mission Analysis Process (6.4.1)
Supply Process (6.1.2)	Project Assessment and Control Process (6.3.2)	Stakeholder Needs and Requirements Definition Process (6.4.2)
Organizational Project-Enabling Processes	Decision Management Process (6.3.3)	Systems/Software Requirements Definition Process (6.4.3)
Life Cycle Model Management Process (6.2.1)	Risk Management Process (6.3.4)	Architecture Definition Process (6.4.4)
Infrastructure Management Process (6.2.2)	Configuration Management Process (6.3.5)	Design Definition Process (6.4.5)
Portfolio Management Process (6.2.3)	Information Management Process (6.3.6)	System Analysis Process (6.4.6)
Human Resource Management Process (6.2.4)	Measurement Process (6.3.7)	Implementation Process (6.4.7)
Quality Management Process (6.2.5)	Quality Assurance Process (6.3.8)	Integration Process (6.4.8)
Knowledge Management Process (6.2.6)		Verification Process (6.4.9)
		Transition Process (6.4.10)
		Validation Process (6.4.11)
		Operation Process (6.4.12)
		Maintenance Process (6.4.13)
		Disposal Process (6.4.14)

Software process descriptions

- ✧ When we describe and discuss processes, we usually talk about the **activities** in these processes such as specifying a data model, designing a user interface, etc. and the ordering of these activities.
- ✧ Process descriptions may also include:
 - **Work/Final Products**, which are the outcomes of a process activity;
 - **Roles**, which reflect the responsibilities of the people involved in the process;
 - **Pre- and post-conditions**, which are statements that are true before and after a process activity has been enacted or a product produced.
 - **Operation Procedure**: a set of written instructions that describes the step-by-step process taken to properly perform a specific activity

Elements of a Software Process

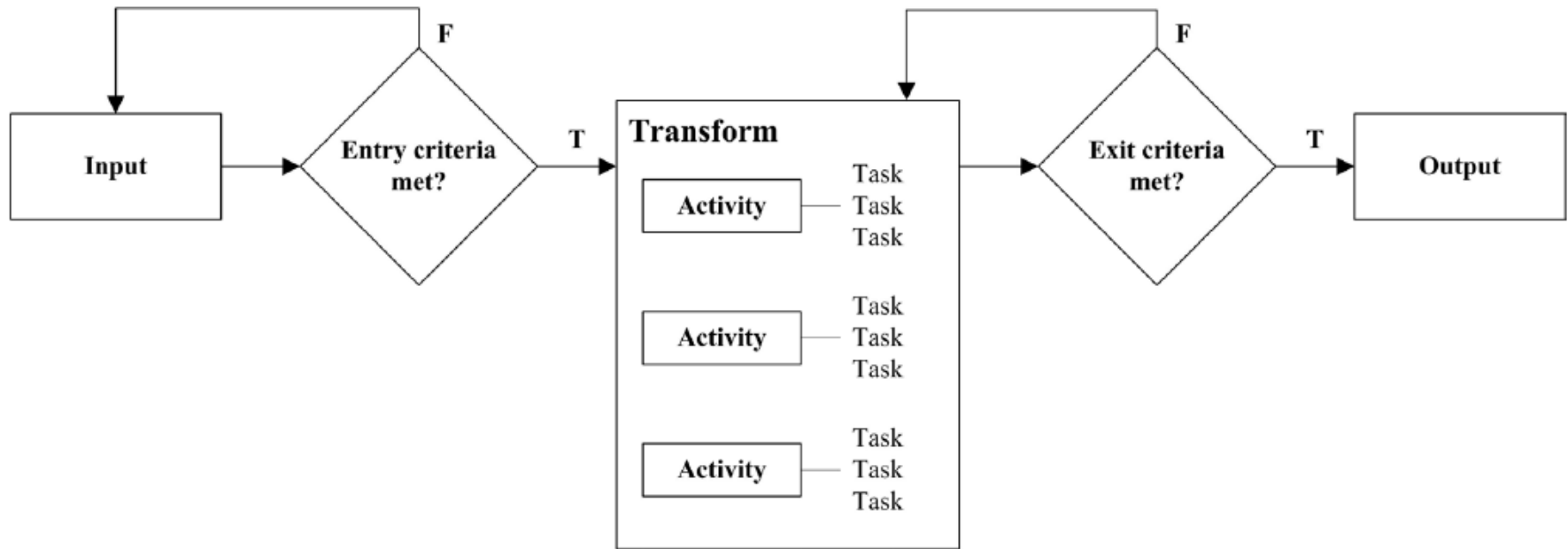
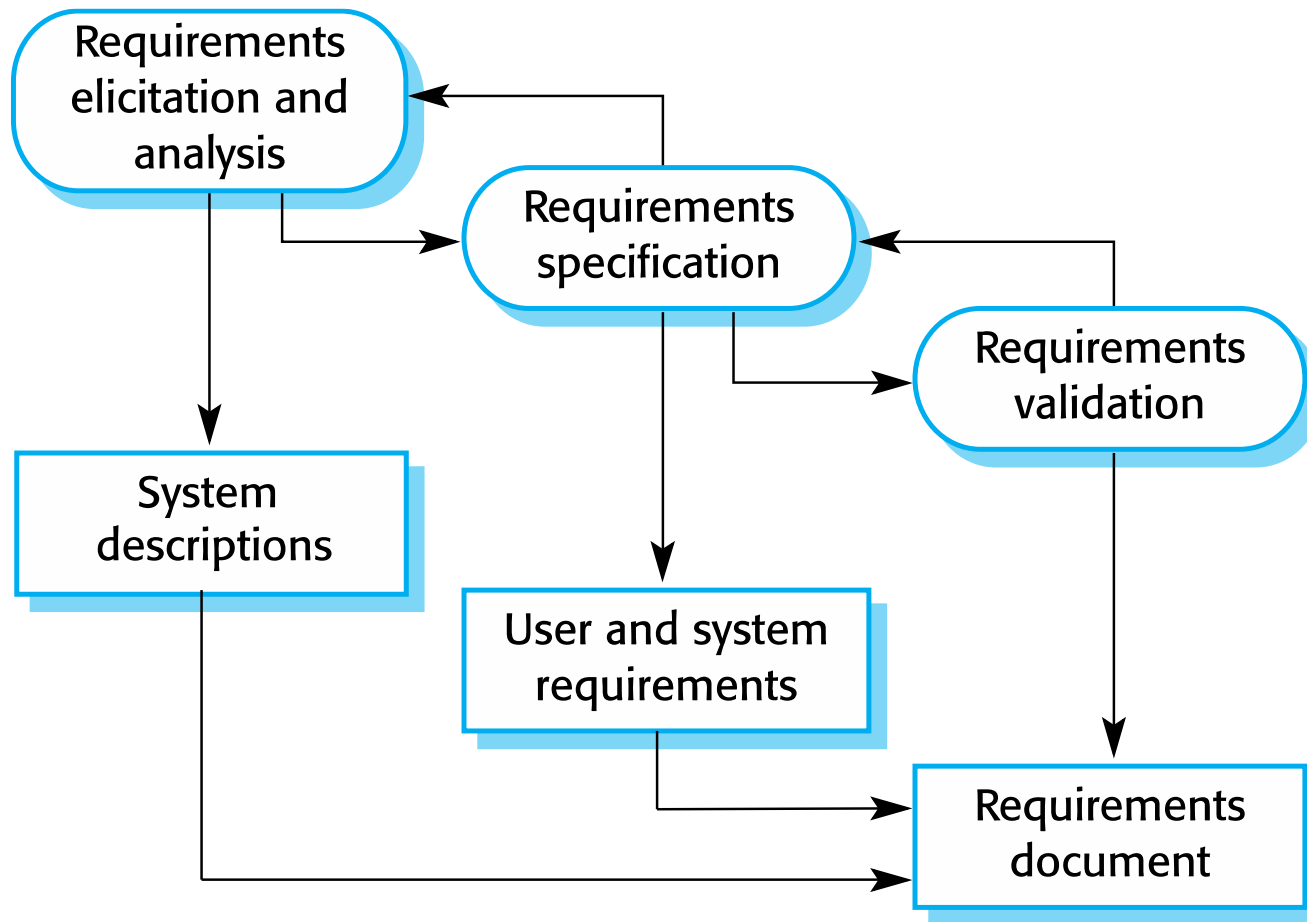


Figure 8.2. Elements of a Software Process

Process activities

- ✧ Real software processes are **inter-leaved sequences** of **technical**, **collaborative** and **managerial** activities with the overall goal of **specifying**, **designing**, **implementing** and **testing** a software system.
- ✧ The **four basic process activities** of **specification**, **development**, **validation** and **evolution** are organized differently in different development processes.
- ✧ For example, in the **waterfall** model, they are organized **in sequence**, whereas in **incremental** development they are **interleaved**.

The requirements engineering process



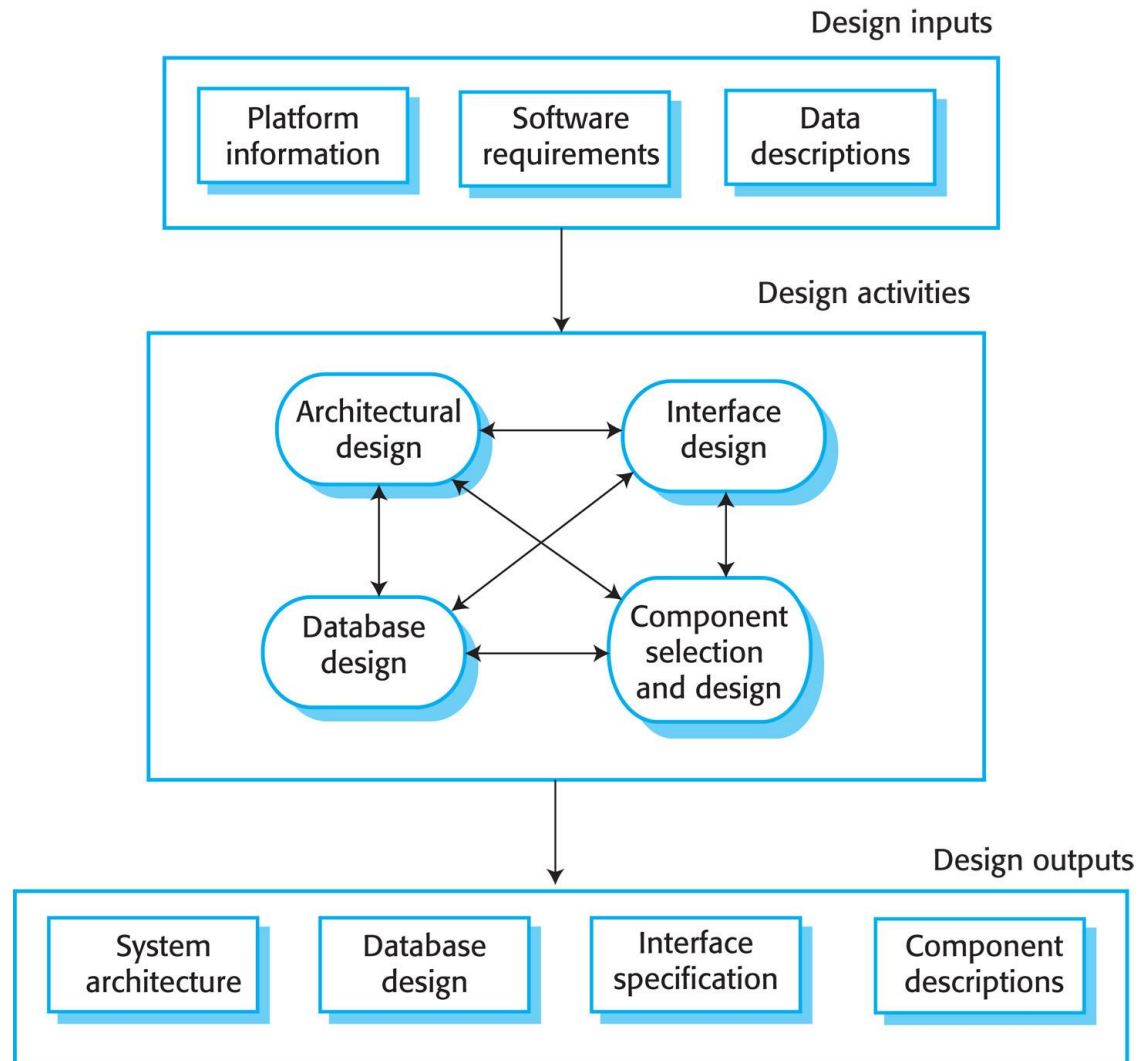
Software specification

- ✧ The process of establishing what services are **required** and the **constraints** on the system's operation and development.
- ✧ Requirements engineering process
 - **Requirements elicitation and analysis**
 - What do the system stakeholders require or expect from the system?
 - **Requirements validation**
 - Checking the validity of the requirements
 - **Requirements specification**
 - Defining the requirements in detail

Software design and implementation

- ✧ The process of **converting** the **system specification** into **an executable system**.
- ✧ Software **design**
 - Design a **software structure** that realises the specification;
- ✧ Implementation
 - Translate this structure into an **executable program**;
- ✧ The activities of design and implementation are closely related and may be inter-leaved.

A general model of the design process



Copyright ©2016 Pearson Education, All Rights Reserved

Design activities

- ✧ **Architectural design**, where you identify the overall structure of the system, the principal components (subsystems or modules), their relationships and how they are distributed.
- ✧ **Database design**, where you design the system data structures and how these are to be represented in a database.
- ✧ **Interface design**, where you define the interfaces between system components.
- ✧ **Component selection and design**, where you search for reusable components. If unavailable, you design how it will operate.

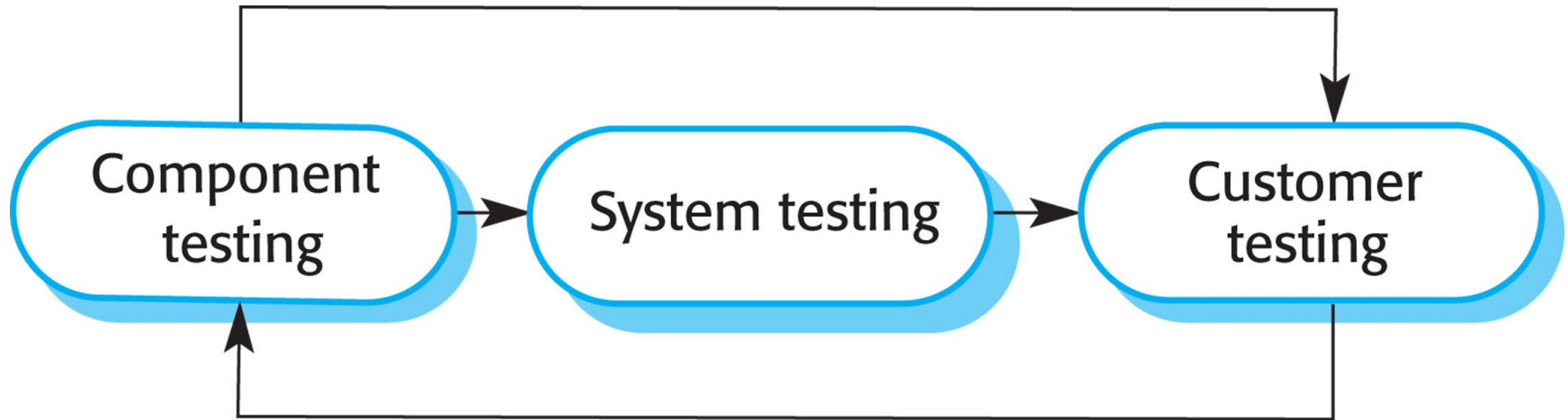
System implementation

- ✧ The software is **implemented** either by developing a program or **programs** or by **configuring** an application system.
- ✧ Design and implementation are interleaved activities for most types of software system.
- ✧ **Programming** is an individual activity with no standard process.
- ✧ **Debugging** is the activity of finding program faults and correcting these faults.

Software validation

- ✧ **Verification** and **validation** (**V & V**) is intended to show that a system **conforms** to its specification and **meets** the **requirements** of the system customer.
- ✧ Involves **checking** and **review** processes and system **testing**.
- ✧ **System testing** involves executing the system with test cases that are derived from the specification of the real data to be processed by the system.
- ✧ **Testing** is the most commonly used **V & V activity**.

Stages of testing



Copyright ©2016 Pearson Education, All Rights Reserved

Testing stages

✧ *Component testing*

- Individual components are tested independently;
- Components may be functions or objects or coherent groupings of these entities.

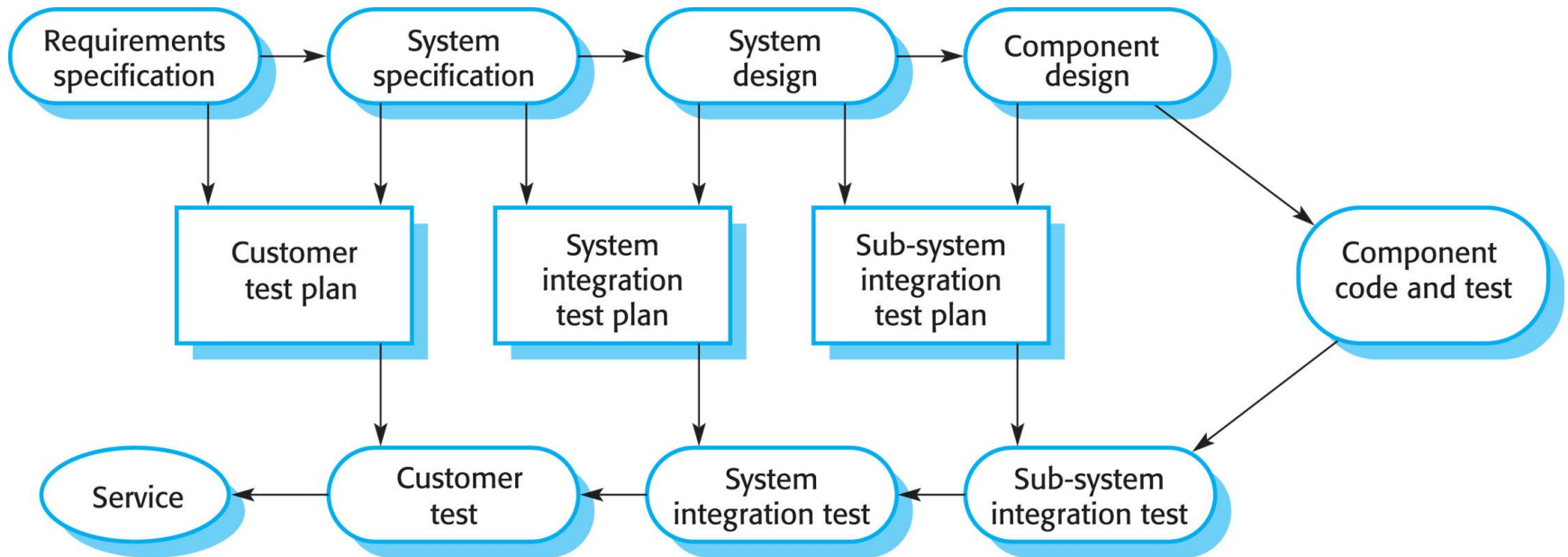
✧ *System testing*

- Testing of the system as a whole. Testing of emergent properties is particularly important.

✧ *Customer testing*

- Testing with customer data to check that the system meets the customer's needs.

Testing phases in a plan-driven software process (**V-model**)

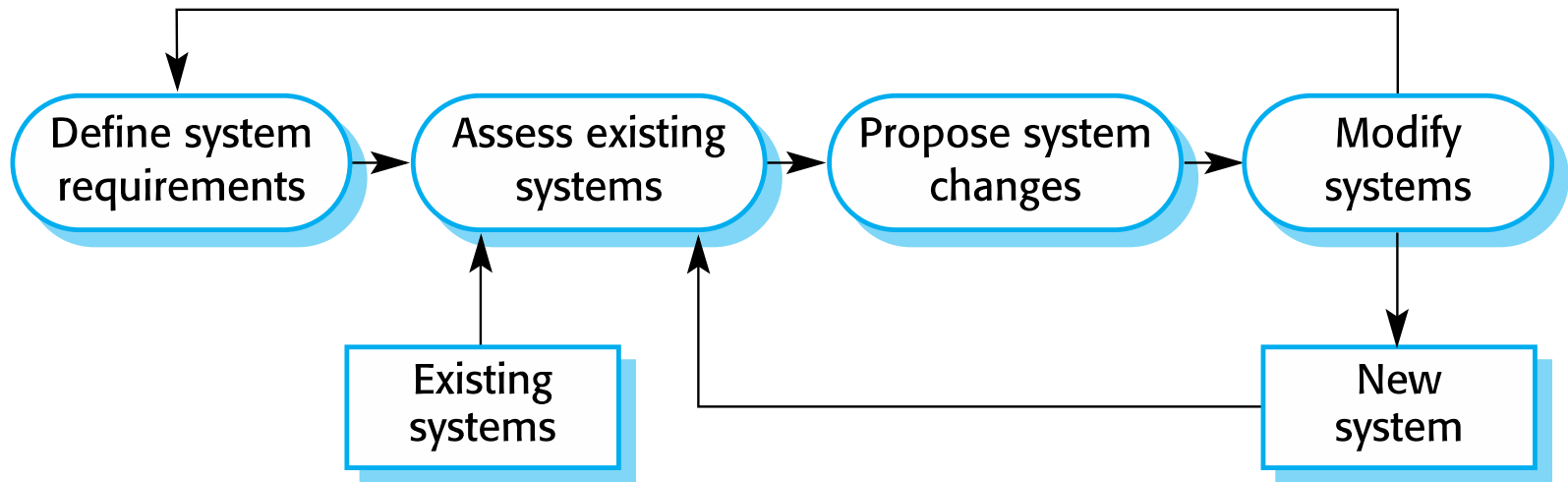


Copyright ©2016 Pearson Education, All Rights Reserved

Software evolution

- ✧ Software is inherently flexible and can change.
- ✧ As requirements change through **changing business circumstances**, the software that supports the business must also evolve and change.
- ✧ Although there has been a demarcation between development and evolution (maintenance) this is increasingly irrelevant as fewer and fewer systems are completely new.

System evolution



Software process models

✧ The **waterfall** model

- Plan-driven model. Separate and distinct phases of specification and development.

✧ **Incremental** development

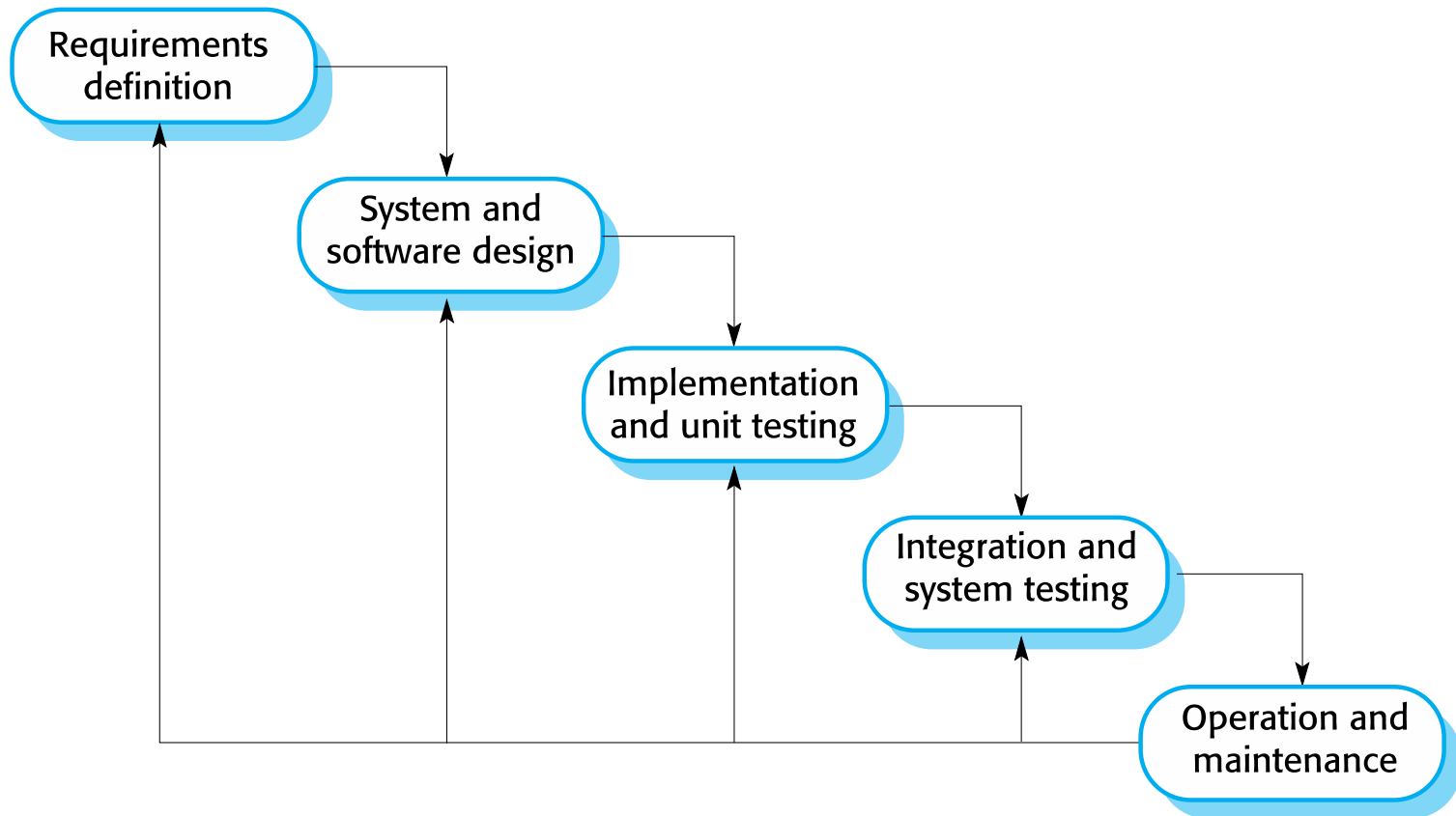
- Specification, development and validation are interleaved. May be plan-driven or agile.

✧ **Integration and configuration**

- The system is assembled from existing configurable components. May be plan-driven or agile.

✧ In practice, **most large systems** are developed using a process that incorporates elements from **all** of these models.

The waterfall model



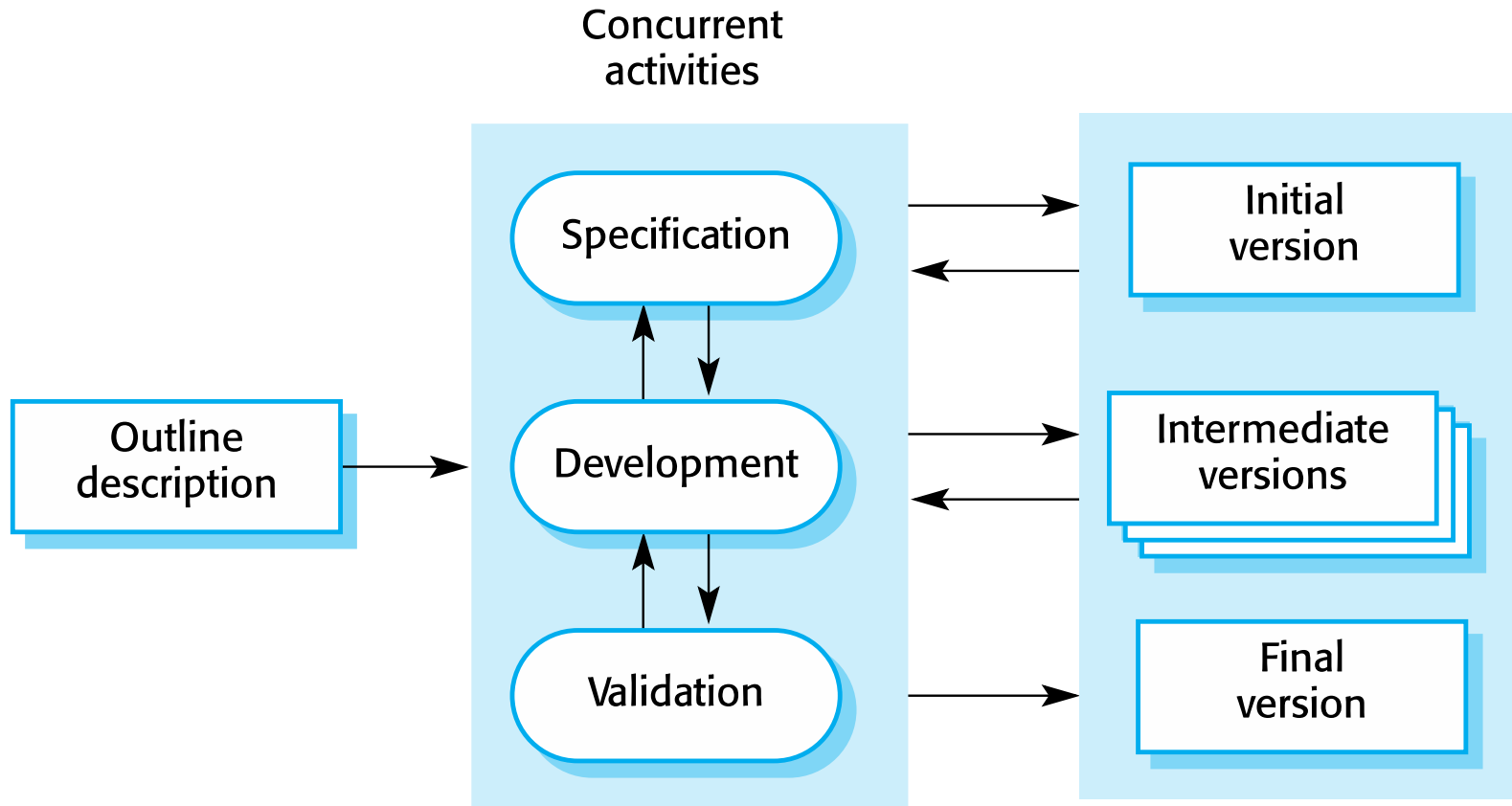
Waterfall model phases

- ✧ There are **separate identified** phases in the waterfall model:
 - Requirements analysis and definition
 - System and software design
 - Implementation and unit testing
 - Integration and system testing
 - Operation and maintenance
- ✧ The main **drawback** of the waterfall model is the **difficulty** of **accommodating change** after the process is underway. In principle, a phase has to be complete before moving onto the next phase.

Waterfall model problems

- ✧ **Inflexible** partitioning of the project into distinct stages makes it **difficult** to respond to changing customer requirements.
 - Therefore, this model is **only appropriate** when the **requirements** are **well-understood** and **changes** will be fairly **limited** during the design process.
 - **Few** business systems have **stable requirements**.
- ✧ The waterfall model is mostly used for **large** systems engineering projects where a system is developed at **several sites**.
 - In those circumstances, the plan-driven nature of the waterfall model **helps coordinate** the work.

Incremental development



Incremental development benefits

- ✧ The **cost** of accommodating **changing** customer requirements is **reduced**.
 - The amount of analysis and documentation that has to be redone is much less than is required with the waterfall model.
- ✧ It is **easier** to get **customer feedback** on the development work that has been done.
 - Customers can comment on demonstrations of the software and see how much has been implemented.
- ✧ More **rapid delivery** and **deployment** of useful software to the customer is possible.
 - Customers are able to use and gain value from the software earlier than is possible with a waterfall process.

Incremental development problems

- ✧ The **process** is not visible.
 - **Managers** need regular deliverables to **measure progress**. If systems are developed quickly, it is not cost-effective to produce documents that reflect every version of the system.
- ✧ System structure tends to **degrade** as new increments are added.
 - Unless time and money is spent on **refactoring** to improve the software, regular change tends to corrupt its structure. Incorporating further software changes becomes increasingly difficult and costly.

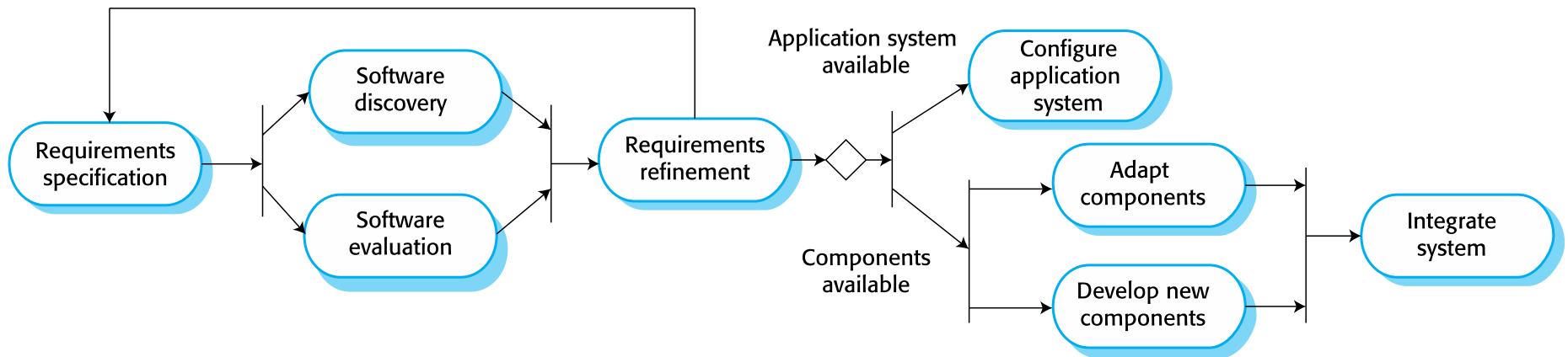
Integration and configuration

- ✧ Based on software reuse where systems are **integrated** from existing components or application systems (sometimes called COTS -Commercial-off-the-shelf) systems).
- ✧ **Reused elements** may be **configured** to **adapt** their behaviour and functionality to a user's requirements
- ✧ **Reuse** is now the **standard approach** for building many types of business system
 - Reuse covered in more depth in Chapter 15.

Types of reusable software

- ✧ Stand-alone application systems (sometimes called **COTS**) that are configured for use in a particular environment.
- ✧ Collections of objects that are developed as a package to be integrated with a component framework such as **.NET** or **J2EE**.
- ✧ **Web services** that are developed according to service standards and which are available for **remote invocation**.

Reuse-oriented software engineering



Key process stages

- ✧ Requirements specification
- ✧ Software discovery and evaluation
- ✧ Requirements refinement
- ✧ Application system configuration
- ✧ Component adaptation and integration

Advantages and disadvantages

- ✧ **Reduced costs** and **risks** as less software is developed from scratch
- ✧ **Faster** delivery and deployment of system
- ✧ But requirements **compromises** are inevitable so system **may not meet real needs** of users
- ✧ **Loss of control** over **evolution** of reused system elements

Plan-driven and agile processes

- ✧ **Plan-driven processes** are processes where all of the process activities are **planned in advance** and **progress** is **measured** against this plan.
- ✧ In **agile** processes, planning is **incremental** and it is **easier** to change the process to **reflect changing** customer requirements.
- ✧ In practice, most **practical** processes **include** elements of **both** plan-driven and agile approaches.
- ✧ There are **no right** or **wrong** software processes.

Coping with change

- ✧ Change is **inevitable** in all large software projects.
 - Business changes lead to new and changed system requirements
 - New technologies open up new possibilities for improving implementations
 - Changing platforms require application changes
- ✧ Change leads to **rework** so the **costs** of change include both rework (e.g. re-analysing requirements) as well as the costs of implementing new functionality

Reducing the costs of rework

- ✧ **Change anticipation**, where the software process includes activities that can anticipate possible changes before significant rework is required.
 - For example, a prototype system may be developed to show some key features of the system to customers.
- ✧ **Change tolerance**, where the process is designed so that changes can be accommodated at relatively low cost.
 - This normally involves some form of **incremental development**. Proposed changes may be implemented in increments that have not yet been developed. If this is impossible, then only a single increment (a small part of the system) may have be altered to incorporate the change.

Coping with changing requirements

- ✧ **System prototyping**, where a version of the system or part of the system is developed quickly to **check** the customer's **requirements** and the **feasibility** of **design** decisions. This approach supports change anticipation.
- ✧ **Incremental delivery**, where system increments are delivered to the customer for **comment** and **experimentation**. This supports both change avoidance and change tolerance.

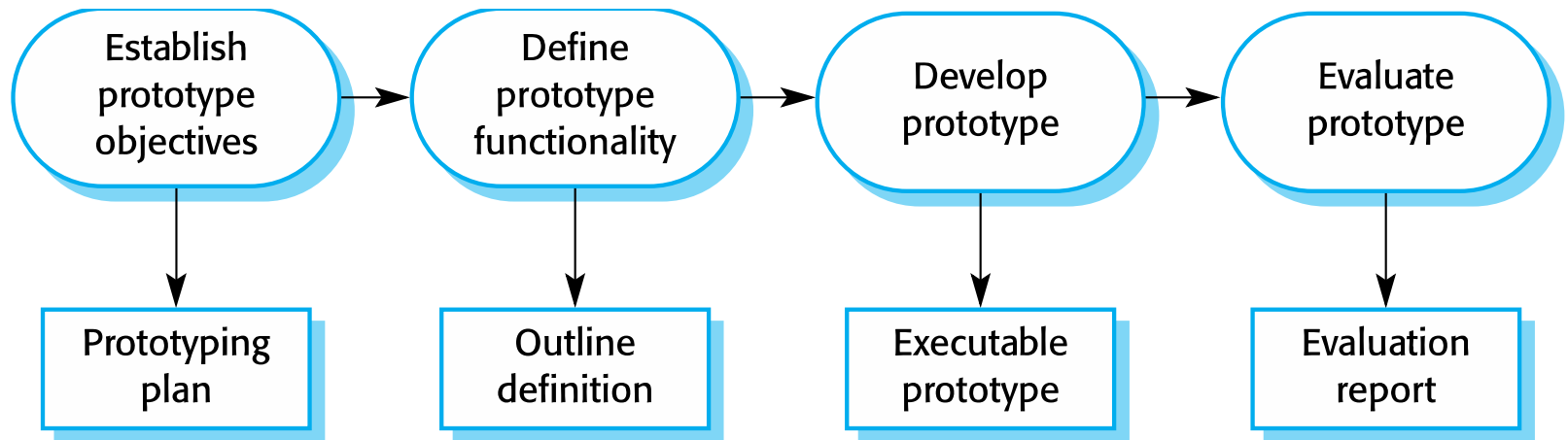
Software prototyping

- ✧ A **prototype** is an initial version of a system used to demonstrate concepts and try out design options.
- ✧ A prototype can be used in:
 - The requirements engineering process to help with **requirements elicitation** and **validation**;
 - In design processes to **explore** options and develop a **UI design**;
 - In the testing process to run back-to-back tests.

Benefits of prototyping

- ✧ **Improved** system **usability**.
- ✧ A **closer match** to users' **real needs**.
- ✧ Improved design quality.
- ✧ Improved maintainability.
- ✧ Reduced development effort.

The process of prototype development



Prototype development

- ✧ May be based on **rapid prototyping** languages or **tools**
- ✧ May involve leaving out functionality
 - Prototype should focus on areas of the product that are **not well-understood**;
 - Error checking and recovery may not be included in the prototype;
 - Focus on **functional** rather than non-functional requirements such as reliability and security

Throw-away prototypes

- ✧ Prototypes should be **discarded** after development as they are not a good basis for a production system:
 - It may be impossible to tune the system to meet non-functional requirements;
 - Prototypes are normally undocumented;
 - The prototype structure is usually degraded through rapid change;
 - The prototype probably will not meet normal organisational quality standards.

Incremental delivery

- ✧ Rather than deliver the system as a single delivery, the development and delivery is broken down into **increments** with each increment delivering part of the required functionality.
- ✧ User requirements are **prioritised** and the **highest priority** requirements are included in early increments.
- ✧ Once the development of an increment is started, the requirements are **frozen** though requirements for **later** increments can continue to evolve.

Incremental development and delivery

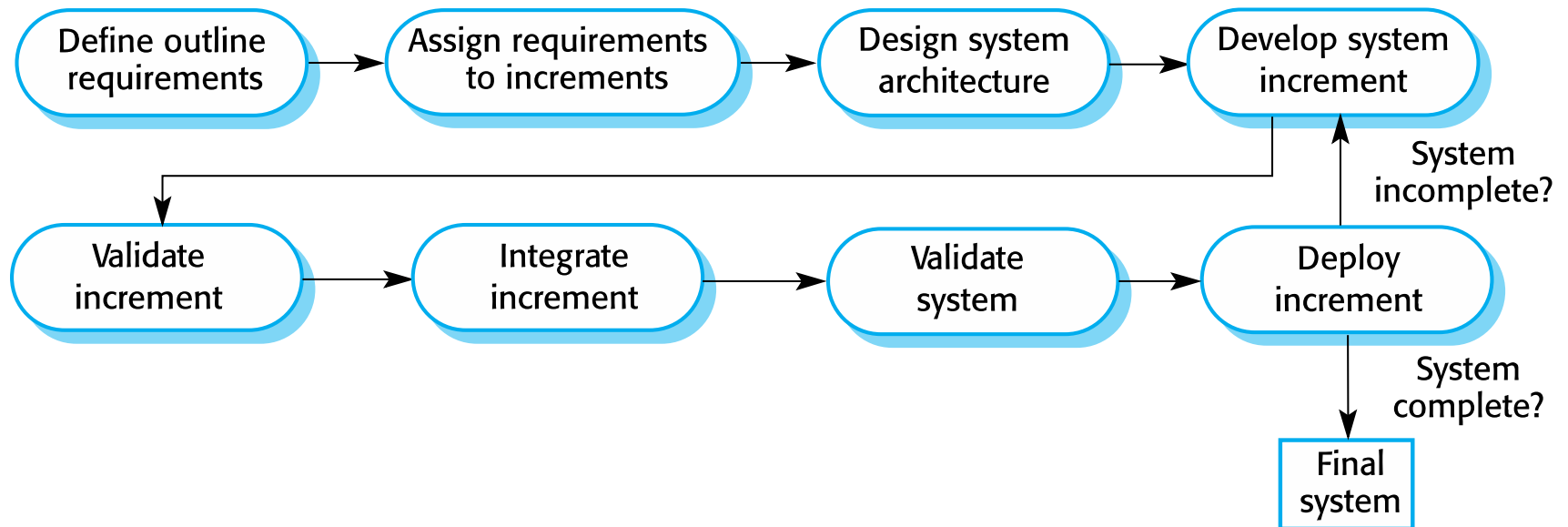
✧ Incremental development

- Develop the system in increments and evaluate each increment before proceeding to the development of the next increment;
- Normal approach used in **agile** methods;
- Evaluation done by user/customer proxy.

✧ Incremental delivery

- **Deploy** an increment for use by end-users;
- More **realistic** evaluation about practical use of software;
- **Difficult** to implement for **replacement systems** as increments have less functionality than the system being replaced.

Incremental delivery



Incremental delivery advantages

- ✧ Customer value can be delivered with each increment so system functionality is available **earlier**.
- ✧ Early increments act as a prototype to help **elicit requirements** for later increments.
- ✧ **Lower risk** of overall project failure.
- ✧ The **highest priority** system services tend to receive the **most testing**.

Incremental delivery problems

- ✧ Most systems require a set of **basic facilities** that are used by different parts of the system.
 - As requirements are not defined in detail until an increment is to be implemented, it can be hard to identify common facilities that are needed by all increments.
- ✧ The essence of **iterative** processes is that the specification is developed in conjunction with the software.
 - However, this **conflicts** with the **procurement** model of many organizations, where the complete system specification is part of the system development contract.

Key points

- ✧ Software processes are the activities involved in producing a software system. Software process models are abstract representations of these processes.
- ✧ General process models describe the organization of software processes.
 - Examples of these general models include the 'waterfall' model, incremental development, and reuse-oriented development.
- ✧ Requirements engineering is the process of developing a software specification.

Key points

- ✧ Design and implementation processes are concerned with transforming a requirements specification into an executable software system.
- ✧ Software validation is the process of checking that the system conforms to its specification and that it meets the real needs of the users of the system.
- ✧ Software evolution takes place when you change existing software systems to meet new requirements. The software must evolve to remain useful.
- ✧ Processes should include activities such as prototyping and incremental delivery to cope with change.

Key points

- ✧ Processes may be structured for iterative development and delivery so that changes may be made without disrupting the system as a whole.
- ✧ The principal approaches to process improvement are agile approaches, geared to reducing process overheads, and maturity-based approaches based on better process management and the use of good software engineering practice.
- ✧ The SEI process maturity framework identifies maturity levels that essentially correspond to the use of good software engineering practice.

工作分解結構圖(WBS)

- ✧ Work Breakdown Structure
- ✧ 為了達成某特定目的(例如開發一軟體系統), 將所需執行的工作項目, 從上而下逐步拆解直到可以分派與執行的工作項目/任務的階層式結構圖
- ✧ 軟體開發生命流程 (Software Development Life Cycle, SDLC)
- ✧ Stage-Process-Activity-Task
- ✧ 階段-流程-作業-工作項目
- ✧ 工作程序 (Procedure): 是為完成某特定任務而制定的工作項目的執行步驟與規範等, 以確保作業能有序進行並達成既定目標

計畫式與敏捷式軟體開發

- ✧ 計畫驅動的流程是指所有流程活動都是事先規劃好，並依據這個計畫來衡量進度
- ✧ 瀑布式軟體開發就是典型的計畫式軟體開發流程之一
- ✧ 在敏捷流程中，計畫是漸進式的，並且更容易根據不斷變化的客戶需求來調整流程
- ✧ Scrum 就是敏捷式軟體開發流程常採用的方法之一
- ✧ 在實務上，大多數實際的流程都是同時包含計畫驅動與敏捷式的元素
- ✧ 沒有絕對正確/錯誤及好/壞的軟體流程

軟體開發生命週期(SDLC)階段/流程

- ✧ 專案規畫
- ✧ 需求分析
- ✧ 系統分析
- ✧ 系統設計
- ✧ 系統開發
- ✧ 系統測試
- ✧ 系統部署
- ✧ 系統維護

各階段目的與主要作業(Activity)

✧ 專案規畫階段

- 目的: 確立專案方向, 避免後期偏差與浪費
- 作業: 專案範疇界定、可行性分析、資源與成本估算

✧ 需求分析階段

- 目的: 確保系統符合使用者與商業需求
- 主要作業: 收集使用者需求、功能與非功能需求文件化 (專案需求說明)

✧ 系統分析階段

- 目的: 將需求轉化為系統規格(軟體需求規格, SRS), 作為設計的基礎, 避免需求誤解
- 作業: 依所採用的分析方法而異

✧ 系統設計階段

- 目的: 提供技術藍圖, 降低開發風險
- 作業: 高階設計 (架構、資料庫)、低階設計 (模組、流程)

結構化系統分析之主要作業 (1/2)

✧ 功能分解 (Functional Decomposition)

- 將系統功能逐層分解成子功能與模組之功能結構圖
- 目的:確保系統功能完整,並利於模組化設計
- 強調「系統如何拆解」

✧ 建立資料流程 (Data Flow)

- 繪製資料在系統中流動的流程圖(DFD), 包含輸入、處理、輸出
- 目的:清楚呈現系統的功能與資料流, 避免遺漏需求
- 強調「資料如何流動」

結構化系統分析之主要作業 (2/2)

✧ 建立實體的關係 (Entity Relation)

- 呈現系統中的實體、屬性及其相互關係與基數(如 1:N)之視覺化結構圖(ERD)
- 目的:釐清實體間的關聯與邏輯架構, 以避免團隊對資料理解的不一致
- 強調「資料如何關聯」

✧ 建立資料字典 (Data Dictionary)

- 以文字表格詳細定義資料元素的名稱、資料型態、業務意涵等
- 目的:提供統一且精確的欄位規範與型態定義, 以降低開發溝通成本, 並確保團隊在資料庫實作時的正確性
- 強調「資料如何定義」

結構化系統設計之主要作業

以「功能與資料流」為核心，強調模組化與流程控制

✧ 模組設計

- 將系統功能分解成模組，定義模組間的介面與流程
- 目的：確保系統功能完整，並利於分工與維護

✧ 資料庫設計

- 依據 ERD 與資料字典DD設計資料表、索引、關聯
- 目的：提供一致且高效的資料存取結構

✧ 流程設計

- 建立流程圖、決策表、決策樹
- 目的：明確描述系統邏輯與控制流程

✧ 介面設計

- 設計使用者介面與模組間介面
- 目的：確保系統易用性與模組間協作

物件導向系統分析之主要作業 (1/2)

✧ 建立功能模型

- 描述系統的功能與資料流，通常透過使用案例描述或活動圖補充
- 目的：確保系統功能完整，並與物件模型、動態模型相互呼應
- 聚焦於「系統提供的功能與流程」

✧ 建立使用案例模型

- 捕捉系統的功能需求，從使用者角度定義系統行為
- 作業：識別系統的主要使用者 (Actors) 與他們與系統的互動 (Use Cases)，繪製使用案例圖
- 聚焦於「系統要做什麼」與「誰使用系統」

物件導向系統分析之主要作業 (2/2)

✧ 建立物件模型

- 識別系統中的主要物件、類別、屬性與方法，建立類別圖
- 目的：將需求轉化為物件結構，提供系統設計的基礎
- 聚焦於「系統由哪些物件組成」

✧ 建立動態模型

- 描述物件之間的互動與狀態變化，例如序列圖、活動圖、狀態圖
- 目的：捕捉系統的行為邏輯與流程，確保物件間的協作正確
- 聚焦於「物件如何互動與狀態如何改變」

物件導向系統設計之主要作業

以「物件與類別」為核心，強調封裝、繼承、多型，並透過互動模型來設計系統

✧ 類別設計

- 定義類別、屬性、方法，建立類別圖
- 目的：將分析階段的物件模型轉化為具體設計藍圖

✧ 物件互動設計

- 建立序列圖、協作圖，描述物件間的訊息傳遞
- 目的：確保物件間的協作正確且符合需求

✧ 狀態設計

- 建立狀態圖，描述物件在不同情境下的行為
- 目的：捕捉系統的動態行為，避免邏輯遺漏

✧ 介面與架構設計

- 定義系統架構、子系統、介面協定
- 目的：確保系統具備良好的擴充性與可維護性

✧ 系統開發階段

- 目的：將設計落實為可執行的軟體
- 主要作業：撰寫程式碼、版本控制、單元測試

✧ 系統測試階段

- 目的：確保軟體品質，排除缺陷
- 作業：單元測試、整合測試、系統測試、使用者驗收測試 (UAT)

✧ 系統部署階段

- 目的：讓軟體可供使用者實際使用
- 作業：上線環境設定、安裝、CI/CD 自動化部署

✧ 系統維護階段

- 目的：確保軟體持續穩定並符合新需求
- 作業：錯誤修正、效能優化、功能更新

作業程序說明書

- ✧ 一份完整的程序說明書 (Standard Operating Procedure, SOP / 作業程序文件)**通常應包含以下主要內容, 讓讀者能「明白為何做、做什麼、誰來做、怎麼做、用什麼、做到什麼程度」

作業程序說明書章節(1/5)

1. 基本資訊

- 文件標題: 程序名稱(例如: 設備操作程序 / 員工報到程序)
- 文件編號: 方便管理與追蹤
- 版本/修訂紀錄: 初版日期、修訂日期、修訂內容
- 適用範圍: 說明此程序適用的部門、對象或情境
- 目的: 為何需要此程序(例如: 確保操作一致性、符合品質/安全要求)

作業程序說明書章節(2/5)

2. 角色與責任

- 執行人員：誰需要依此程序操作
- 主管/審核人員：誰負責監督、批准或檢查
- 支援單位：需要配合的其他人員或部門

作業程序說明書章節(3/5)

3. 作業流程(核心內容)

- 流程圖 / 步驟說明: 以圖表或條列方式呈現, 清楚指出先後順序
- 步驟細節
- 每一步要做什麼
- 如何執行(方法、標準、工具)
- 注意事項與判斷標準
- 所需資源: 儀器、設備、表單、材料、軟體系統等

作業程序說明書章節(4/5)

4. 品質與安全規範

- 品質標準：執行完成後應達成的檢核條件。
- 安全注意事項：潛在風險、緊急應變措施、防護裝備。
- 相關法規 / 標準依據：如 ISO、GMP、公司政策等。

5. 文件與紀錄

- 表單與紀錄：需要填寫的表格、報告或系統紀錄。
- 保存方式：紀錄保存期限、存放位置、責任人員。

作業程序說明書章節 (5/5)

6. 附錄 / 支援資訊

- 專業術語定義。
- 參考文件。
- 圖示、範例或常見問題 FAQ。